

MODULE 1

Git & GitHub

De zéro à une utilisation professionnelle

Objectif du parcours : construire progressivement les compétences nécessaires pour devenir ingénieur MLOps.

Méthode : comprendre → pratiquer → se tromper → corriger → refaire sans aide.

Page 2 — Comment utiliser ce module

Ce document est notre support de cours. Nous n'allons pas simplement lire les pages : nous allons les transformer en pratique.

Règle de la formation : je peux t'expliquer une page précise, puis te donner un exercice basé dessus. Tu essaies d'abord seul.

Quand je te dirai par exemple « regarde la page 14 », tu pourras ouvrir ce PDF et retrouver exactement la notion concernée.

À la fin du module, tu devras être capable de créer un dépôt, faire des commits propres, travailler avec des branches, pousser vers GitHub, récupérer un projet, résoudre un conflit simple et comprendre le rôle de GitHub dans un workflow professionnel.

Page 3 — Git et GitHub : la différence

Git est un système de contrôle de version distribué. Il garde l'historique des modifications d'un projet.

GitHub est une plateforme en ligne qui héberge des dépôts Git et ajoute des fonctions de collaboration.

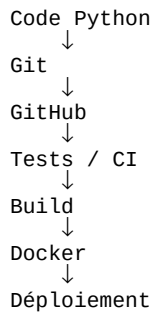
Git = outil de versioning
GitHub = plateforme de collaboration autour de Git

Analogie : Git est le système qui enregistre les versions de ton projet ; GitHub est un espace distant où tu peux stocker et partager ce dépôt.

Important : Git peut fonctionner sans GitHub. GitHub ne remplace pas Git.

Page 4 — Pourquoi Git est indispensable en MLOps

Un projet MLOps évolue constamment : code, scripts, modèles, configuration, pipelines et infrastructure. Il faut pouvoir savoir ce qui a changé, quand, et pourquoi.



Git devient donc une pièce centrale de la chaîne de production. Tu ne dois pas seulement savoir taper des commandes : tu dois comprendre le workflow.

Page 5 — Installer et vérifier Git

Après installation de Git, vérifie qu'il répond :

```
git --version
```

Configure ensuite ton identité Git. Elle sera associée à tes commits :

```
git config --global user.name "Ton Nom"  
git config --global user.email "ton@email.com"
```

Pour vérifier la configuration :

```
git config --list
```

Ne mets jamais un mot de passe, une clé API ou un secret dans un dépôt Git.

Page 6 — Repository

Un **repository** (dépôt) est l'espace dans lequel Git suit l'évolution d'un projet.

Pour transformer un dossier existant en dépôt Git :

```
cd mon-projet  
git init
```

Git crée alors un dossier caché **.git** qui contient les informations nécessaires à l'historique.

Attention : ne supprime pas ou ne modifie pas manuellement **.git** si tu ne sais pas exactement ce que tu fais.

Page 7 — Le cycle de travail Git

```
Fichier modifié
↓
Working directory
↓ git add
Staging area
↓ git commit
Repository local
↓ git push
Repository distant (GitHub)
```

Les trois zones à retenir :

- **Working directory** : les fichiers sur lesquels tu travailles.
- **Staging area** : les changements sélectionnés pour le prochain commit.
- **Repository** : l'historique enregistré par Git.

Page 8 — git status

C'est l'une des commandes que tu utiliseras le plus.

```
git status
```

Elle indique notamment les fichiers modifiés, les fichiers déjà placés dans la staging area et les fichiers non suivis.

Réflexe professionnel : avant de faire un commit, lance git status.

Exercice mental : si tu modifies trois fichiers mais que tu ne veux committer que deux d'entre eux, Git doit te permettre de sélectionner précisément ces deux fichiers.

Page 9 — git add

Ajouter un fichier à la staging area :

```
git add fichier.py
```

Ajouter plusieurs fichiers :

```
git add fichier1.py fichier2.py
```

Ajouter tout le contenu modifié du dossier courant :

```
git add .
```

Attention : git add . est pratique mais tu dois comprendre ce qu'il sélectionne avant de l'utiliser.

Page 10 — git commit

Un commit enregistre un état précis du projet dans l'historique Git.

```
git commit -m "Ajoute la fonction de calcul de moyenne"
```

Un bon message de commit explique clairement le changement.

Évite les messages comme :

```
git commit -m "test"  
git commit -m "modif"  
git commit -m "aaaa"
```

Préférer un message qui décrit l'intention du changement.

Page 11 — Lire l'historique

```
git log
```

Version compacte :

```
git log --oneline
```

L'historique permet de retrouver les commits précédents et leurs identifiants.

Concept important : Git ne sauvegarde pas seulement « le fichier actuel ». Il construit un historique permettant de revenir sur les versions précédentes.

Page 12 — Modifier puis vérifier

Workflow minimal :

```
# 1. Modifier le code
# 2. Vérifier
git status

# 3. Sélectionner
git add fichier.py

# 4. Vérifier encore
git status

# 5. Enregistrer
git commit -m "Décrit le changement"
```

Ce cycle doit devenir un automatisme.

Page 13 — git diff

Pour voir les changements qui ne sont pas encore dans la staging area :

```
git diff
```

Pour examiner les changements déjà placés dans la staging area :

```
git diff --staged
```

Ces commandes sont extrêmement utiles pour éviter de committer accidentellement une modification.

Page 14 — Les branches

Une branche permet de travailler sur une ligne d'évolution séparée sans modifier directement la branche principale.

```
git branch
git switch -c feature-login
```

Exemple :

```
main
■
■■■ feature-login
■
■■■ feature-dashboard
```

À retenir : une branche n'est pas « un deuxième projet ». C'est une autre ligne de travail dans le même historique.

Page 15 — Changer de branche

```
git switch main  
git switch feature-login
```

Créer puis basculer sur une nouvelle branche :

```
git switch -c feature-api
```

Ancienne commande souvent rencontrée :

```
git checkout -b feature-api
```

Tu dois connaître checkout car tu le rencontreras dans beaucoup de tutoriels, mais nous privilégierons switch pour les changements de branche.

Page 16 — Merge

Le merge permet d'intégrer les changements d'une branche dans une autre.

```
git switch main  
git merge feature-login
```

Le sens est important : tu te places d'abord sur la branche qui doit recevoir les changements.

Page 17 — Les conflits

Un conflit peut apparaître lorsque Git ne peut pas déterminer automatiquement quelle version conserver.

```
<<<<<< HEAD
version A
=====
version B
>>>>>> feature-login
```

Méthode générale :

- Lire le conflit.
- Décider quelle version garder ou comment combiner les deux.
- Supprimer les marqueurs de conflit.
- Ajouter le fichier corrigé avec git add.
- Terminer l'opération de merge avec le commit demandé par Git.

Page 18 — GitHub et remote

Un remote est une référence vers un dépôt distant.

```
git remote -v
```

Ajouter un remote :

```
git remote add origin URL_DU_DEPOT
```

Puis envoyer la branche :

```
git push -u origin main
```

Le nom origin est une convention courante : ce n'est pas une obligation technique.

Page 19 — clone, pull, fetch

Récupérer un projet complet :

```
git clone URL_DU_DEPOT
```

Récupérer les changements et les intégrer dans ta branche courante :

```
git pull
```

Récupérer les informations distantes sans intégrer automatiquement les changements :

```
git fetch
```

Différence essentielle : fetch récupère ; pull récupère puis intègre selon la configuration du dépôt.

Page 20 — .gitignore

Le fichier **.gitignore** indique à Git quels fichiers ou dossiers ne doivent généralement pas être suivis.

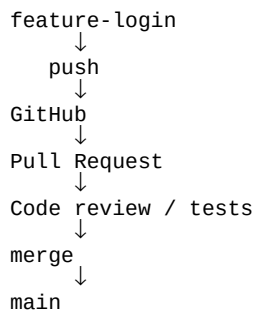
```
# Exemple
__pycache__/  
.venv/  
.env  
*.log
```

Pour un projet Python, cela évite notamment de versionner des fichiers générés ou des secrets locaux.

Règle de sécurité : ne mets jamais de clé API, mot de passe ou secret dans Git.

Page 21 — GitHub : Pull Request

Une Pull Request (PR) sert à proposer l'intégration d'une branche dans une autre et à permettre une revue du code.



Dans une équipe, la PR est un élément essentiel du travail collaboratif.

Page 22 — README, Issues et Releases

README : explique le projet, son installation, son utilisation et ses prérequis.

Issues : permettent de suivre bugs, tâches et améliorations.

Releases/Tags : permettent d'identifier des versions importantes.

v1.0.0
v1.1.0
v2.0.0

Ces éléments sont importants pour passer d'un simple projet étudiant à un projet maintenable.

Page 23 — Bonnes pratiques de commits

- Un commit doit représenter un changement cohérent.
- Évite les énormes commits qui mélangent dix fonctionnalités.
- Utilise des messages précis.
- Committe régulièrement.
- Ne committe pas de secrets.
- Vérifie git diff et git status avant les commits importants.

Exemples :

```
feat: ajoute l'API de prédiction  
fix: corrige le calcul de moyenne  
docs: met à jour le README  
test: ajoute les tests du service de scoring
```

Page 24 — Git et Python : notre premier projet

Nous utiliserons un petit projet Python pour apprendre Git en pratique.

```
mlops-training/  
  ■■■ src/  
  ■   ■■■ calcul.py  
  ■■■ tests/  
  ■   ■■■ test_calcul.py  
  ■■■ README.md  
  ■■■ .gitignore  
  ■■■ requirements.txt
```

Tu vas créer ce projet, faire plusieurs modifications, créer des branches, faire des commits et publier le dépôt sur GitHub.

Page 25 — Workflow professionnel simplifié

1. `git clone`
2. `git switch -c feature/...`
3. coder
4. `git status`
5. `git diff`
6. `git add`
7. `git commit`
8. `git push`
9. Pull Request
10. Review + tests
11. Merge
12. Retour sur main
13. `git pull`

Ce workflow deviendra ensuite la base de notre CI/CD et de notre parcours MLOps.

Page 26 — Ce que tu dois savoir faire sans aide

- Créer un dépôt Git.
- Comprendre working directory, staging area et repository.
- Faire un commit propre.
- Lire l'historique.
- Comparer des changements.
- Créer et changer de branche.
- Fusionner une branche.
- Comprendre et résoudre un conflit simple.
- Cloner un dépôt GitHub.
- Faire push / pull / fetch.
- Créer un .gitignore adapté.
- Comprendre une Pull Request.
- Expliquer la différence entre Git et GitHub.

Page 27 — Exercices du module

Exercice 1 — Repository local

Crée un dossier nommé formation-python, initialise Git, ajoute un fichier Python et fais ton premier commit.

Exercice 2 — Historique

Modifie le fichier, fais un deuxième commit, puis affiche l'historique avec `git log --oneline`.

Exercice 3 — Branche

Crée une branche feature-calcul, ajoute une fonction Python, committe-la puis fusionne-la dans main.

Exercice 4 — GitHub

Crée un dépôt GitHub, connecte ton dépôt local et pousse ton projet.

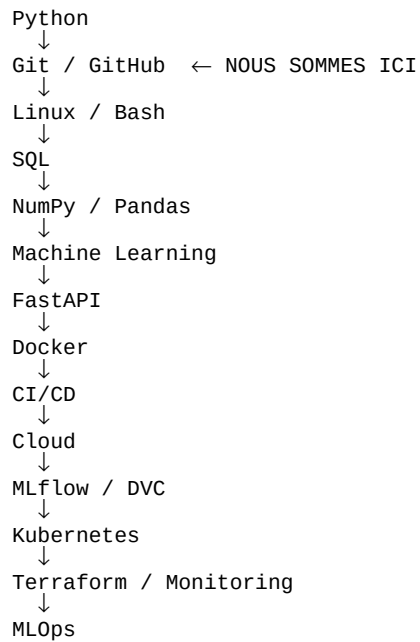
Page 28 — Défi final

Sans copier une suite de commandes donnée par quelqu'un, réalise ce scénario :

- Créer un projet Python vide.
- Initialiser Git.
- Faire un premier commit.
- Créer une branche de fonctionnalité.
- Ajouter une fonctionnalité Python.
- Faire deux commits propres.
- Publier la branche sur GitHub.
- Créer une Pull Request.
- Fusionner la branche.
- Mettre à jour ton dépôt local.

Critère de réussite : tu dois pouvoir expliquer à voix haute ce que fait chaque commande et pourquoi tu l'utilises.

Page 29 — Git dans notre parcours MLOps



Git n'est donc pas une matière isolée. Il devient la colonne vertébrale de notre future chaîne de développement et de déploiement.

Page 30 — Validation du module

Avant de considérer Git comme maîtrisé, tu dois réussir trois choses :

1. **Compréhension** : expliquer Git, GitHub, repository, staging, commit, branch, merge et remote.
2. **Pratique** : réaliser un projet complet local → GitHub sans tutoriel pas à pas.
3. **Autonomie** : diagnostiquer les erreurs courantes à partir de git status, git log et des messages de Git.

Quand ces trois objectifs sont atteints, nous passerons à la suite du parcours parallèle : **Linux/Bash**, tout en continuant notre progression principale en Python.

Page 31 — Mini glossaire

Terme	Signification
Git	Système de contrôle de version
GitHub	Plateforme d'hébergement et de collaboration Git
Repository	Dépôt contenant l'historique du projet
Commit	Enregistrement d'un état du projet
Staging area	Zone préparant les changements pour un commit
Branch	Ligne de développement séparée
Merge	Intégration d'une branche dans une autre
Remote	Référence vers un dépôt distant
Clone	Copie d'un dépôt distant en local
Push	Envoi de commits vers un dépôt distant
Pull	Récupération et intégration des changements distants
Fetch	Récupération des informations distantes sans intégration immédiate
Pull Request	Proposition de changement destinée à être revue puis fusionnée